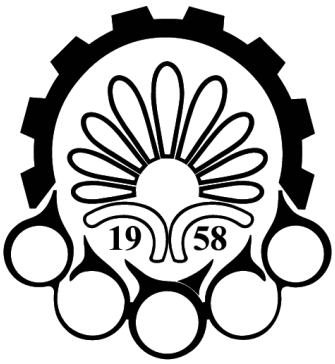


Amirkabir University of Technology
(Tehran Polytechnic)

گزارش پروژه دوم پایگاه داده

دانشجو : ماهان فتح اله پورکامی

شماره دانشجویی : ۴۰۳۳۱۰۳۴

بهار ۴۰۵

استاد درس : دکتر شهریاری

Phase1-normalization

با توجه به نمودار ER نهایی ، موجودیت‌ها و صفات کلیدی آن‌ها را به صورت زیر فرموله می‌کنیم:

۱.مشتری و آدرس ها :

- صفات مشتری :

Buyer : {<u>ID</u> , first_name , last_name , phone , email , register_date , status }

از آنجا که شناسه ، کلید کاندید (اصلی) است داریم :

$$\mathbf{ID \rightarrow first_name , last_name , phone , email , register_date , status}$$

همچنین بدلیل اینکه شماره تلفن و ایمیل هر شخص منحصر به فرد است داریم :

$$\mathbf{phone \rightarrow ID}$$
$$\mathbf{email \rightarrow ID}$$

در هر ۳ FD در سمت چپ یک کلید کاندیدا واقع شده است، بنابراین این رابطه در سطح BCNF نرمال است.

- صفات آدرس ارسال :

Address : {<u>Address_ID</u> , City , street , N.O ,Buyer_ID}

از آنجا که شناسه آدرس، کلید کاندید (اصلی) است داریم :

Address_ID → city , street , N.O ,Buyer_ID

در تنها FD موجود نیز سمت چپ یک کلید کاندیدا واقع شده، پس این رابطه نیز در سطح BCNF نرمال است.

---

۲. فروشندگان :

- صفات فروشندگان :

Seller : {eco_num , bus_name , phone , email , is_confirmed, register_date }

با توجه به اینکه کد اقتصادی هر فروشنده منحصر به فرد است ، پس eco_num کلید کاندید (اصلی) این رابطه است .

eco_num →bus_name , phone , email , is_confirmed , register_date

همچنین با توجه به اینکه شماره تماس و ایمیل هر شخص یکتا است داریم :

$$phone \rightarrow eco_num$$

$$email \rightarrow eco_num$$

در هر ۳ FD چون سمت چپ یک کلید کاندیدا قرار دارد پس این رابطه در سطح BCNF نرمال است.

۳.محصولات و موجودی انبار :

- صفات محصول :

Product : {<u>ID</u> , details ,active , price}

همچنین شناسه هر محصول یکتا است ، پس شناسه ، کلید کاندید (اصلی) این رابطه است .

$$ID \rightarrow details , active , price$$

با توجه به اینکه در تنها FD موجود ، در سمت چپ یک کلید کاندید وجود دارد، بنابراین این رابطه نیز در سطح BCNF نرمال است .

- صفات موجودی انبار :

Storage_quantity : {<u>Product ID</u>, quantity_count , location , last_update_date , warning_number }

با توجه به اینکه این موجودیت با محصول رابطه یک به یک دارد بنابراین ، شناسه محصول کلید کاندید (اصلی) این موجودیت ضعیف محسوب میشود.

$Product$_ID →quantity_count , location ,last_update_date ,warning_number

در تنها FD داده شده ، در سمت چپ یک کلیدکاندید قرار داده شده است ، بنابراین این رابطه نیز در سطح BCNF نرمال است .

۴ .دسته‌بندی‌ها(Category) **:**

- صفات دسته بندی :

Category : {<u>ID</u> , Name , Parent_Category_ID}

همچنین برای وابستگی تابعی داریم :

$$ID \rightarrow Name\ , Parent_Category_ID$$

$$Name\ \rightarrow ID\ , Parent_Category_ID$$

با توجه به اینکه در سمت چپ هر دو FD یک کلید کاندیدا قرار داده شده (نام و شناسه هر دسته بندی منحصر به فرد است) ، بنابراین این رابطه نیز در سطح BCNF نرمال است .

---

۵ .سفارش‌ها و آیتم‌های سفارش :

- صفارت سفارش :

Order : {<u>ID</u>,register_date,send_cost,discount,status,Buyer_ID,Address_ID}

وابستگی های تابعی :

$$ID \rightarrow register_date , total , send_cost , discount , status , Buyer_ID, Address_ID$$

در تنها FD موجود در این رابطه سمت چپ یک کلید کاندید قرار دارد ، پس این رابطه در سطح BCNF نرمال است.

- صفات آیتم سفارش :

Order_Item : { <u>Order_item_ID</u> , Order_ID , Product_ID , quantity , unit_price }

وابستگی های تابعی :

$$Order_item_ID \rightarrow Order_ID , Product_ID , quantity , unit_price$$

صفت موجود در سمت چپ یک کلید کاندیدا است پس رابطه در سطح BCNF نرمال است.

نکته محاسباتی :قیمت نهایی هر سطر حاصل‌ضرب تعداد در قیمت واحد است و یک ویژگی محاسباتی است، به همین دلیل برای حفظ نرمال‌سازی و عدم افزونگی، آن را ذخیره نمی‌کنیم یا به صورت داینامیک محاسبه می‌کنیم.

۶. پرداخت‌ها و ارسال‌ها :

- صفات پرداخت :

Pay : {<u>code</u>,fee,pay_way,status,time,Order_ID}

وابستگی های تابعی :

$$code \rightarrow company\ ,send_time\ ,recieve_time\ ,status\ ,Order_ID$$

صفت موجود در سمت چپ یک کلید کاندیدا است پس رابطه در سطح BCNF نرمال است.

- صفات ارسال :

{<u>code</u>,company,send_time,receive_time,status,Order_ID}

وابستگی های تابعی :

$$code \rightarrow company, send_time\ ,recieve_time\ ,status, Order_ID$$

صفت موجود در سمت چپ یک کلید کاندیدا است پس رابطه در سطح BCNF نرمال است.

۷ .کدهای تخفیف:

صفات کد تخفیف :

{<u>code</u>,type,value,min_price,max_usage}

وابستگی های تابعی :

$$code \rightarrow type, value, min_price, max_usage$$

صفت موجود در سمت چپ یک کلید کاندیدا است پس رابطه در سطح BCNF نرمال است.

فرآیند نرمال سازی :

- ✓ **جداسازی اطلاعات مشتری از آدرس‌ها** :به دلیل اینکه هر مشتری می‌تواند چندین آدرس داشته باشد، قرار دادن آدرس در جدول مشتری باعث تکرار اطلاعات مشتری می‌شد .لذا جدول Buyerو Addressبه صورت دو جدول مجزا با رابطه 1:Nدر آمدند .در هر دو جدول تمام وابستگی‌های تابعی به شکل $X \rightarrow Y$ هستند که در آن‌ها x حتماً یک کلید کاندید است.
- ✓ **جداسازی محصول از موجودی انبار** :برای جلوگیری از قفل شدن جدول پرکاربرد محصول در هنگام به‌روزرسانی مداوم موجودی، صفات انبارداری به جدول

مستقل Storage_Quantity منتقل شدند .کلید اصلی این جدول همان Product_ID است و چون رابطه 1:1 است، شرایط BCNF کاملاً برقرار است .

- ✓ **جداسازی سفارش از آیتم‌های سفارش** :هر سفارش شامل چندین محصول است . برای جلوگیری از ایجاد آنومالی، اطلاعات کلی سفارش (مشتری، تاریخ، آدرس ارسال، تخفیف) در جدول Orderو جزئیات هر کالا (تعداد، قیمت واحد خرید) در جدول واسط Order_itemذخیره می‌شوند .تمام FD شناسایی شده در این جداول در سطح BCNF قرار دارند .
- ✓ **جداسازی پرداخت‌ها و ارسال‌ها** :از آنجا که پرداخت‌ها و ارسال‌ها دارای چرخه‌های وضعیت متفاوتی از خود سفارش هستند و ممکن است چند بار تلاش برای پرداخت صورت گیرد، به جداول مجزای Payو Postتفکیک شدند تا از تجمع فیلدهای تهی (Null) در جدول سفارش جلوگیری شود .هر دو جدول بر اساس کلیدهای اصلی خود در سطح BCNF نرمال هستند .

Phase4-Index

ایندکس ترکیبی idx_order_buyer_date **:**

این شاخص به صورت ترکیبی روی buyer_idو register_date DESC ساخته شده است. این کار باعث می‌شود کوئری ۲ که ۵ سفارش اخیر یک مشتری را می‌خواهد، بدون نیاز به اسکن کامل جدول Table و عملیات سنگین Sort در حافظه، مستقیماً و در زمان O(log n) نتایج مرتب‌شده را از درخت B-Tree استخراج کند.

ایندکس‌های کلید خارجی (idx_product_seller , idx_order_address **و** idx_order_item_product)**:**

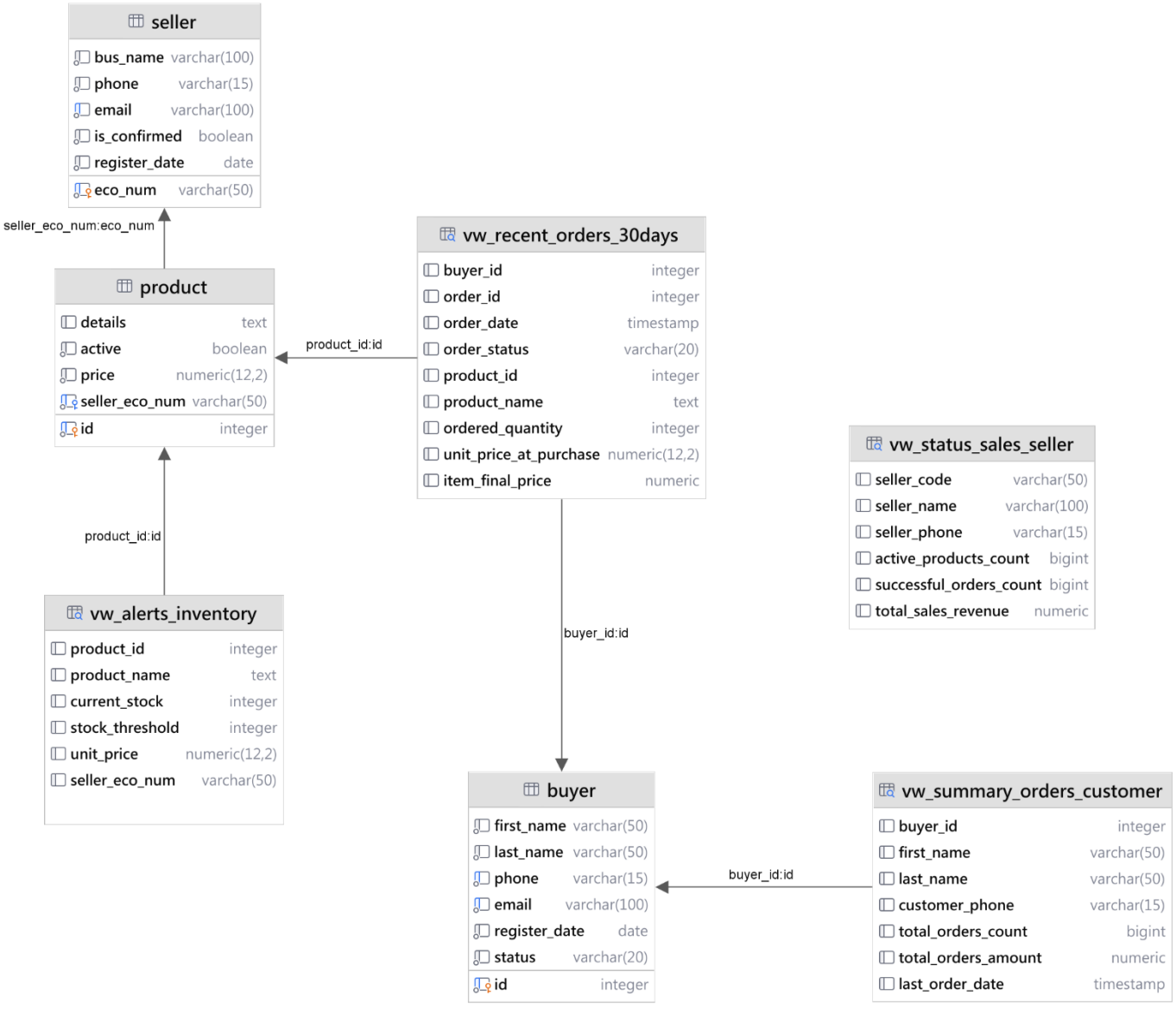
پایگاه داده PostgreSQL به طور خودکار روی کلیدهای خارجی ایندکس نمی‌سازد. ساخت این ایندکس‌ها برای کوئری‌هایی که نیاز به اتصالات مکرر (JOIN) بین جداول اصلی و واسط دارند (مانند محاسبات فروش ماهانه فروشندگان) سرعت الحاق جداول را به شدت افزایش می‌دهد.

ایندکس فیلتر وضعیت (idx_pay_status , idx_post_status) **:**

در سیستم‌های فروشگاهی، بررسی تراکنش‌های معلق یا خطا خورده و همچنین پیگیری مرسوله پستی ، یک پروسه مداوم است. ساخت ایندکس روی ستون status

جدول Pay,Post کمک می‌کند تا فورا و بدون گشتن تمام تراکنش‌های موفق تاریخی، و یا تمامی ارسال ها جزییات ردیف‌های دلخواه را پیدا کند.

Phase5-view


seller
bus_name varchar(100)
phone varchar(15)
email varchar(100)
is_confirmed boolean
register_date date
eco_num varchar(50)
seller_eco_num:eco_num
product
details text
active boolean
price numeric(12,2)
seller_eco_num varchar(50)
id integer
product_id:id
vw_recent_orders_30days
buyer_id integer
order_id integer
order_date timestamp
order_status varchar(20)
product_id integer
product_name text
ordered_quantity integer
unit_price_at_purchase numeric(12,2)
item_final_price numeric
vw_status_sales_seller
seller_code varchar(50)
seller_name varchar(100)
seller_phone varchar(15)
active_products_count bigint
successful_orders_count bigint
total_sales_revenue numeric
product_id:id
vw_alerts_inventory
product_id integer
product_name text
current_stock integer
stock_threshold integer
unit_price numeric(12,2)
seller_eco_num varchar(50)
buyer_id:id
buyer
first_name varchar(50)
last_name varchar(50)
phone varchar(15)
email varchar(100)
register_date date
status varchar(20)
id integer
buyer_id:id
vw_summary_orders_customer
buyer_id integer
first_name varchar(50)
last_name varchar(50)
customer_phone varchar(15)
total_orders_count bigint
total_orders_amount numeric
last_order_date timestamp

Phase6-procedure

---

۱. توضیح گام‌به‌گام روال pr_place_order :

این Procedure برای ایجاد یک سفارش جدید به همراه آیتم‌های آن، بررسی وضعیت خریدار، هماهنگی آدرس، کنترل موجودی انبار، اعمال کوپن تخفیف (درصدی یا مبلغی) و به‌روزرسانی نهایی مبلغ کل طراحی شده است.

- **گام ۱: اعتبارسنجی خریدار و وضعیت حساب :**

ابتدا با یک شرط EXISTSبررسی می‌شود که شناسه خریدار وجود داشته باشد. در گام بعد، وضعیت حساب او از جدول Buyer استخراج شده و تنها در صورتی که وضعیت برابر با 'active' باشد، اجازه ادامه کار داده می‌شود.

- **گام ۲: انطباق آدرس با مشتری :**

Procedure با جستجو در جدول Address اطمینان حاصل می‌کند که p_address_id ارسالی، دقیقاً متعلق به همین خریدار (buyer_id) باشد تا از ثبت آدرس اشتباه جلوگیری شود.

- **گام ۳: هماهنگی لیست محصولات و تعداد درخواست :**

بررسی می‌شود که طول آرایه کالاها با آرایه تعداد درخواستی برابر باشد؛ در غیر این صورت خطا صادر می‌شود.

- **گام ۴: ایجاد سطر اولیه در جدول سفارش :**

یک رکورد در جدول "Order"با مبالغ موقت و وضعیت پیش‌فرض 'pending'درج شده و کلید اصلی آن (id) با دستور RETURNING برای مراحل بعدی ذخیره می‌شود.

- **گام ۵: حلقه پیمایش کالاها و مدیریت انبار فیزیکی :**

یک حلقه بر اساس طول آرایه کالاها اجرا می‌شود که در هر گردش کارهای زیر را انجام می‌دهد:

- بررسی کالا :قیمت و وضعیت فعال بودن کالا از جدول Productاستخراج می‌شود. اگر کالا غیرفعال باشد، خطا صادر خواهد شد.
- کنترل انبار :ستون quantity_countاز جدول Storage_Quantity بررسی می‌شود. اگر موجودی کمتر از تعداد درخواستی کاربر باشد، تراکنش متوقف می‌شود.
- ثبت آیتم :کالا با قیمت واحدِ آن زمان، در جدول Order_item درج می‌شود.

- کسر از انبار :موجودی کالا در Storage_Quantity کاهش یافته و زمان last_update_date به زمان جاری سیستم تغییر می‌کند.
- محاسبه جمع جزء :حاصل‌ضرب تعداد در قیمت به متغیر v_subtotal اضافه می‌شود.

- **گام ۶: بررسی و اعمال هوشمند کوپن تخفیف :**

اگر کد تخفیف ارسال شده باشد، اطلاعات آن از جدول Couponاستخراج می‌شود. در صورت عبور از شرط حداقل خرید :

- اگر نوع کوپن **'percent'**باشد، درصد مشخص شده روی مبلغ v_subtotal اعمال می‌شود.
- اگر نوع کوپن **'amount'**باشد، به اندازه مبلغ ثابت کوپن از فاکتور کسر می‌شود.

- **گام ۷: نهایی‌سازی سفارش :**

هزینه ارسال به فاکتور اضافه شده، تخفیف کسر می‌شود و رکوردی که در گام سوم ایجاد شده بود، با مبلغ کل نهایی (total) و میزان تخفیف کسر شده (discount)به‌روزرسانی می‌شود.

---

۲ .توضیح گام‌به‌گام pr_register_payment :

این procedure مسئولیت درج تراکنش‌های مالی در جدول پرداخت‌ها و مدیریت تغییر وضعیت سفارش به پرداخت‌شده را بر عهده دارد.

- **گام ۱: احراز وجود سفارش :**

بررسی می‌کند که شناسه سفارش (p_order_id) در جدول **"Order"** وجود داشته باشد.

- **گام ۲: استخراج وضعیت و مبلغ مصوب سفارش :**

مبلغ کل فاکتور (total) و وضعیت فعلی سفارش از جدول اصلی واکشی می‌شود.

- **گام ۳: کنترل گیت‌های منطقی :**

اگر سفارش لغو شده باشد، اجازه پرداخت داده نمی‌شود. همچنین اگر وضعیت پرداخت ارسالی 'successful' باشد اما سفارش از قبل پرداخت شده یا در فرآیند لجستیک باشد، جلوی عملیات تکراری گرفته می‌شود.

- **گام ۴: درج در جدول پرداخت با ستون‌های فیزیکی دیتابیس :**

یک سطر جدید در جدول Payدرج می‌شود. در این بخش، کد رهگیری بانکی در ستون code(کلید اصلی)، مبلغ واریزی در ستون feeو زمان تراکنش در ستون timeقرار می‌گیرند.

- **گام ۵: تطبیق مالی و ارتقای وضعیت سفارش :**

اگر پرداخت موفقیت‌آمیز باشد('successful') ، پروسجر چک می‌کند که مبلغ واریز شده (p_fee) کمتر از مبلغ واقعی سفارش نباشد. در صورت صحت، ستون status جدول "Order"به 'paid'تغییر می‌کند.

---

۳ .توضیح گام‌به‌گام روال pr_cancel_order :

این procedure فرآیند ابطال سفارش‌های ثبت‌شده را پیش از ارسال فیزیکی مدیریت کرده و کالاها را به چرخه انبار بازمی‌گرداند.

- **گام ۱: بررسی وجود سفارش :**

مانند روال‌های قبلی، ابتدا از وجود داشتن سفارش در سیستم اطمینان حاصل می‌کند.

- **گام ۲: بررسی امکان منع لغو سفارش‌های ارسالی :**

وضعیت فعلی سفارش بررسی می‌شود. طبق قانون سیستم، اگر سفارش در وضعیت‌های نهایی ارسال یعنی 'shipped'(تحویل به پست) یا 'delivered'(تحویل به مشتری) باشد، یا از قبل لغو شده باشد، عملیات با صادر کردن خطای مناسب متوقف می‌شود.

- **گام ۳: حلقه بازگشت کالا به موجودی انبار :**

Procedure با اجرای یک حلقه روی تمام ردیف‌های جدول Order_item که مربوط به این سفارش هستند، تک‌تک محصولات را پیدا کرده و تعداد لغوشده را به ستون quantity_count در جدول Storage_Quantity اضافه می‌کند تا انبار دوباره شارژ شود. در همین حال، زمان last_update_date انبار نیز به‌روزرسانی می‌شود.

- **گام ۴: تغییر وضعیت به cancelled :**

در گام نهایی، ستون statusسفارش مورد نظر در جدول "Order"به وضعیت نهایی 'cancelled'تغییر پیدا می‌کند.

Phase7-transaction

Query | Query History

```
1  -- tr_place_order: Successful
2  DO $$
3  DECLARE
4      v_order_id INT;
5  BEGIN
6      CALL pr_place_order(2, 3, NULL, ARRAY[3], ARRAY[1], v_order_id);
7      RAISE NOTICE 'Test 1 (Success): Order placed successfully. Generated ID: %', v_order_id;
8  EXCEPTION WHEN OTHERS THEN
9      RAISE NOTICE 'Test 1 Failed: %', SQLERRM;
10 END $$;
```

Data Output | Messages | Notifications

```
NOTICE:  Test 1 (Success): Order placed successfully. Generated ID: 13
DO

Query returned successfully in 20 msec.
```

- تست اول تراکنش اول (صحیح)

Query | Query History

```
1  -- tr_place_order: Unsuccessful case 2
2  DO $$
3  BEGIN
4      CALL pr_place_order(1, 1, NULL, ARRAY[1], ARRAY[1000], NULL);
5      RAISE NOTICE 'Test 3 (Success): Order placed successfully.';
6  EXCEPTION WHEN OTHERS THEN
7      RAISE NOTICE 'Test 3 (Expected Failure): Transaction rolled back. Reason: %', SQLERRM;
8  END $$;
```

Data Output | Messages | Notifications

```
NOTICE:  Test 3 (Expected Failure): Transaction rolled back. Reason: procedure parameter "p_generated_order_id" is
DO

Query returned successfully in 23 msec.
```

- تست دوم تراکنش اول (خطا)

Query | Query History

```
-- tr_place_order: Unsuccessful case 1
DO $$
BEGIN
    CALL pr_place_order(999, 1, NULL, ARRAY[1], ARRAY[1], NULL);
    RAISE NOTICE 'Test 2 (Success): Order placed successfully.';
EXCEPTION WHEN OTHERS THEN
    RAISE NOTICE 'Test 2 (Expected Failure): Transaction rolled back. Reason: %', SQLERRM;
END $$;
```

Data Output | Messages | Notifications

```
NOTICE:  Test 2 (Expected Failure): Transaction rolled back. Reason: procedure parameter "p_generated_order_id" is an
DO

Query returned successfully in 20 msec.
```

- تست سوم تراکنش اول (خطا)

Query | Query History

```
-- tr_payment_order: Successful case
DO $$
BEGIN
    CALL pr_register_payment(9, 800000.00, 'ZarinPal', 'TRC-SUCCESS-009', 'successful');
    RAISE NOTICE 'Test 4 (Success): Payment registered and order updated successfully.';
EXCEPTION WHEN OTHERS THEN
    RAISE NOTICE 'Test 4 Failed: %', SQLERRM;
END $$;
```

Data Output | Messages | Notifications

```
NOTICE:  Test 4 (Success): Payment registered and order updated successfully.
DO

Query returned successfully in 21 msec.
```

- تست اول تراکنش دوم (صحیح)

Query Query History

```
1  -- tr_payment_order: Unsuccessful case 1
2  DO $$
3  BEGIN
4      CALL pr_register_payment(12, 1000.00, 'Saman', 'TRC-FAIL-012', 'successful');
5      RAISE NOTICE 'Test 5 (Success): Payment registered successfully.';
6  EXCEPTION WHEN OTHERS THEN
7      RAISE NOTICE 'Test 5 (Expected Failure): Transaction rolled back. Reason: %', SQLERRM;
8  END $$;
```

Data Output Messages Notifications

```
NOTICE:  Test 5 (Expected Failure): Transaction rolled back. Reason: ERROR: Insufficient payment fee. Expected: 85025000.00, Got: 1000.00
DO

Query returned successfully in 23 msec.
```

- تست دوم تراکنش دوم (خطا)

Query Query History

```
1   -- tr_payment_order: Unsuccessful case 2
2   DO $$
3   BEGIN
4       UPDATE "Order" SET status = 'shipped' WHERE id = 1;
5
6       CALL pr_register_payment(1, 1200000.00, 'Mellat', 'TRC-FAIL-001', 'successful');
7       RAISE NOTICE 'Test 6 (Success): Payment registered successfully.';
8   EXCEPTION WHEN OTHERS THEN
9       RAISE NOTICE 'Test 6 (Expected Failure): Transaction rolled back. Reason: %', SQLERRM;
10  END $$;
```

Data Output Messages Notifications

```
NOTICE:  Test 6 (Expected Failure): Transaction rolled back. Reason: ERROR: Order is already paid.
DO

Query returned successfully in 22 msec.
```

- تست سوم تراکنش دوم (خطا)

```
Query  Query History
1  -- tr_cancel_ord: Successful case
2  DO $$
3  BEGIN
4      CALL pr_cancel_order(12);
5      RAISE NOTICE 'Test 7 (Success): Order canceled and stock reverted successfully.';
6  EXCEPTION WHEN OTHERS THEN
7      RAISE NOTICE 'Test 7 Failed: %', SQLERRM;
8  END $$;
```

```
Data Output  Messages  Notifications
NOTICE:  Test 7 (Success): Order canceled and stock reverted successfully.
DO

Query returned successfully in 21 msec.
```

- تست اول تراکنش سوم (صحیح)

```
Query  Query History
1  -- tr_cancel_ord: Unsuccessful case 1
2  DO $$
3  BEGIN
4      UPDATE "Order" SET status = 'shipped' WHERE id = 2;
5
6      CALL pr_cancel_order(2);
7      RAISE NOTICE 'Test 8 (Success): Order canceled successfully.';
8  EXCEPTION WHEN OTHERS THEN
9      RAISE NOTICE 'Test 8 (Expected Failure): Transaction rolled back. Reason: %', SQLERRM;
10 END $$;
11
```

```
Data Output  Messages  Notifications
NOTICE:  Test 8 (Expected Failure): Transaction rolled back. Reason: ERROR: Order already shipped or delivered.
DO

Query returned successfully in 20 msec.
```

- تست دوم تراکنش سوم (خطا)

Query Query History

```
1   -- tr_cancel_ord: Unsuccessful case 2
2   DO $$
3   BEGIN
4       CALL pr_cancel_order(9999);
5       RAISE NOTICE 'Test 9 (Success): Order canceled successfully.';
6   EXCEPTION WHEN OTHERS THEN
7       RAISE NOTICE 'Test 9 (Expected Failure): Transaction rolled back. Reason: %', SQLERRM;
8   END $$;
```

Data Output Messages Notifications

```
NOTICE:  Test 9 (Expected Failure): Transaction rolled back. Reason: ERROR: Order not found.
DO

Query returned successfully in 19 msec.
```

- تست سوم تراکنش سوم (خطا)